esade

Recommender Systems

Matrix Factorization

Prof. Marc Torrens, PhD
www.marctorrens.com
Associated Professor @ Esade
Board Member @ Medidedalia
Cofounder @ Strands

Matrix Factorisation: Intuition

esade

User Item Matrix

User	Movie A	Movie B	Movie C	Movie D	Movie E
U1	5	4	?	1	?
U2	4	?	5	1	?
U3	?	1	?	5	4
U4	1	?	2	5	?

- The matrix is sparse (most users have rated only few items)
- Content-base filtering needs good metadata

Matrix factorisation tries to fill the missing values by learning hidden patterns

Matrix Factorisation: Intuition

esade

User Item Matrix

User	Movie A	Movie B	Movie C	Movie D	Movie E
U1	5	4	?	1	?
U2	4	?	5	1	?
U3	?	1	?	5	4
U4	1	?	2	5	?

- The matrix is sparse (most users have rated only few items)
- Content-base filtering needs good metadata

Matrix factorisation tries to fill the missing values by learning hidden patterns

Matrix Factorisation: Intuition

esade

If we would have factors describing items and users things would be very easy:

Movie	Action factor	Romance factor
Die Hard	0.9	0.1
The Notebook	0.1	0.9
Titanic	0.3	0.8
Mad Max	1.0	0.0

User	Action preference	Romance preference
U1	0.8	0.2
U2	0.2	0.9

Each user has a vector:

$$p_u$$

Each item has a vector:

$$q_i$$

The predicted rating is the dot product:

$$\hat{r}_{ui} = p_u \cdot q_i$$

For example:

$$p_u = [0.8, 0.2]$$

$$q_i = [0.9, 0.1]$$

$$\hat{r}_{ui} = 0.8 \times 0.9 + 0.2 \times 0.1 = 0.74$$

Matrix Factorisation: Intuition

esade

But… we do not have these factors. Can we learn them from the rating matrix?

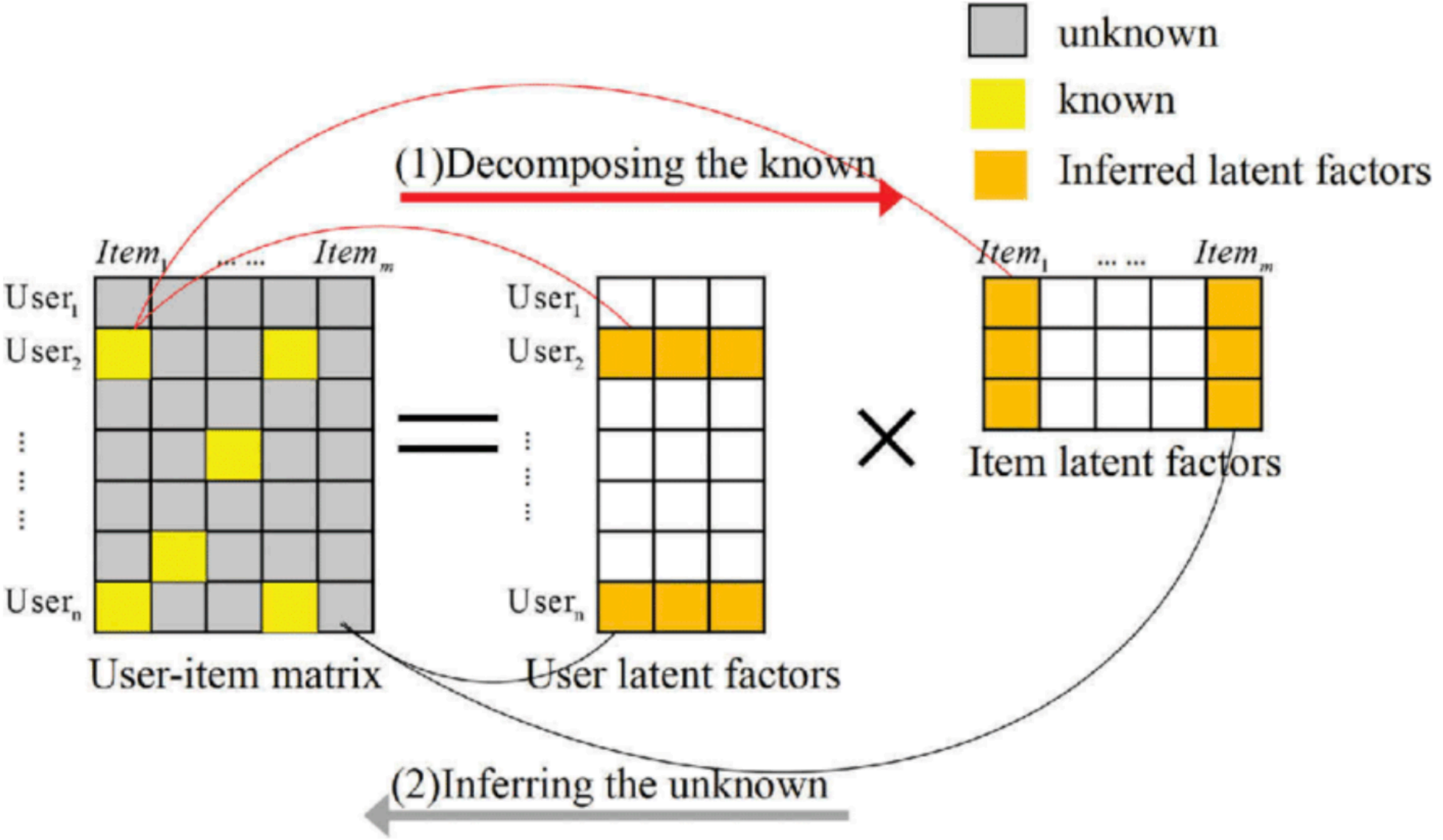
Matrix Factorisation allows us to discover those hidden factors.

These hidden factors are hard to interpret. A hidden factor could combine genre, popularity, mood, decade, etc. They are not *natural,* they are the result of the data.

$$R \approx PQ^T$$

- R = user-item rating matrix
- P = user-factor matrix
- Q = item-factor matrix
- Q^T = transpose of item-factor matrix

Matrix Factorisation



Matrix Factorisation: the algorithm

esade

Machine Learning allows us to discover those hidden factors.

$$R \approx PQ^T$$

- R = user-item rating matrix
- P = user-factor matrix
- Q = item-factor matrix
- Q^T = transpose of item-factor matrix

The algorithm **learns P and Q** by **minimising prediction error** on known ratings. So, the objective function of our ML algorithm is:

$$\min_{P,Q} \sum_{(u,i) \in K} (r_{ui} - p_u \cdot q_i)^2$$

- K = known ratings
- r_{ui} = true rating
- $p_u \cdot q_i$ = predicted rating

Matrix Factorisation: the algorithm

esade

The learning algorithm is quite straightforward:

1. Start with random user and movie vectors.
2. Predict a known rating using the dot product.
3. Compare prediction with the real rating.
4. Compute the error.
5. Slightly adjust the user and movie vectors to reduce the error.
6. Repeat many times until predictions improve.

The algorithm learns hidden user preferences and hidden movie characteristics by repeatedly correcting its mistakes.

Matrix Factorisation: the algorithm

esade

Different implementations:

Singular Value Decomposition
(sklearn.decompoistion.TrucatedSVD)

Non-negative Matrix Factorisation
(sklearn.decompoisiton.NMF)

Alternating Least Squares
(implicit.als.AlternatingLeastSquares)

Matrix Factorisation: limitations

1. Latent factors are hard to interpret.

The model may learn factors, but we do not always know what they mean (hidden factors).

Matrix Factorisation: limitations

esade

2. Cold-start problem

Matrix factorisation needs interactions (known ratings in the rating matrix)

It dos not work for new users or new items

Matrix Factorisation: limitations

esade

3. Can reinforce popularity bias

Popular items have many ratings, so their factors may be better learned and more often recommended.

Matrix Factorisation: limitations

esade

4. Needs tuning

The algorithm requires hyper parameters such as:

- Number of latent factors
- Learning rate
- Regularisation
- Number of iterations

Matrix Factorisation: takeaway

esade

- Matrix Factorisation compresses the user-item matrix into a small number of hidden factors.
- It is powerful because it can discover latent patterns in behaviour
- It depends on rating data

Matrix Factorisation: in the industry...

esade

THE NETFLIX PRIZE

A case study in recommender systems, matrix factorization, and crowdsourced innovation

$1M
10% better than Cinematch

2006 launch

100M+ ratings

2009 winner

MF + ensembles

Sources: Netflix Prize Rules; Koren, Bell & Volinsky (2009); Wired (2009, 2012).

Why Netflix launched a public challenge

A recommendation problem became an innovation strategy

Business problem

Better recommendations were expected to improve customer satisfaction, retention, and DVD queue quality.

Competition design

Netflix released a large anonymized rating dataset and offered a prize to improve its Cinematch algorithm by at least 10%.

crowd R&D

Core strategic idea

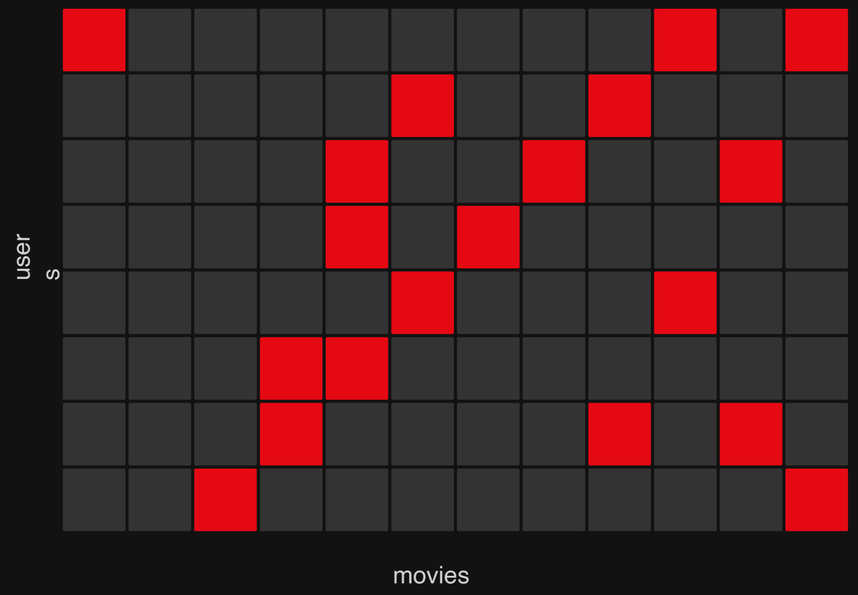
Turn a difficult internal ML problem into a global benchmark where thousands of teams could experiment, publish ideas, and combine methods.

Sources: Netflix Prize Rules; Harvard Business School case abstract; Wired coverage.

The benchmark: a giant sparse rating matrix

The public challenge was framed as rating prediction

Known ratings



Goal
Predict missing ratings as accurately as possible

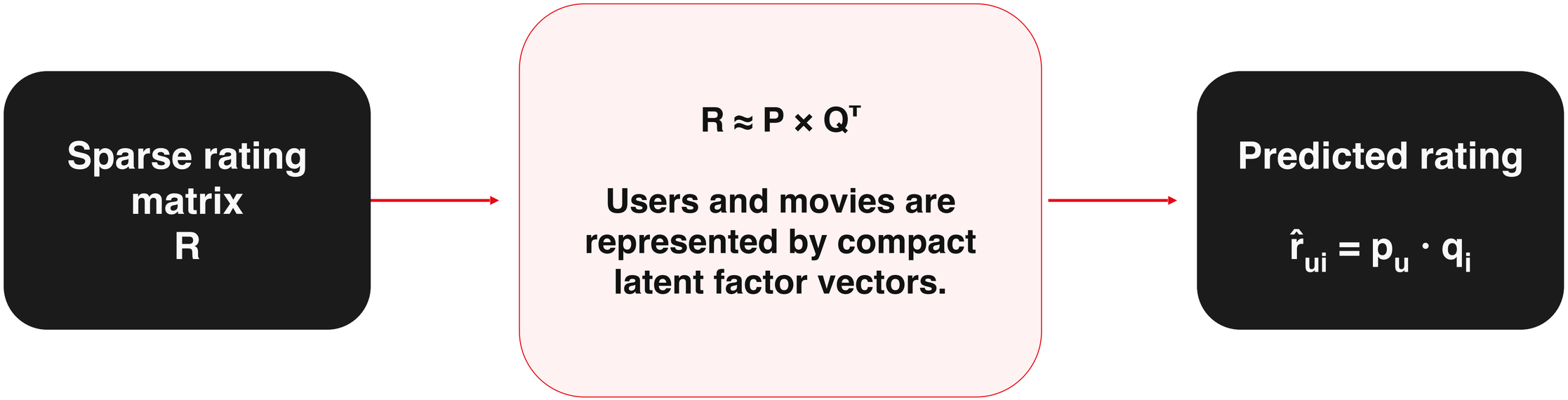
Metric
RMSE on hidden test ratings

RMSE focused the competition on one clear number — but that number captured only one view of recommendation quality.

Sources: Netflix Prize Rules; Netflix Prize dataset descriptions summarized in Koren et al. (2009).

Why matrix factorization mattered

From nearest neighbours to latent user–movie factors



Latent factors can capture hidden structure such as genre, mood, popularity, niche taste, or rating behavior — without manually defining those attributes.

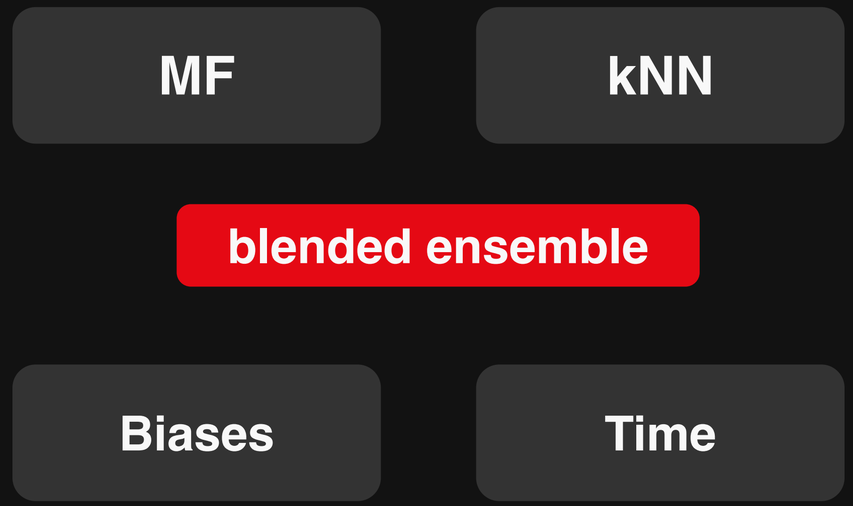
Source: Koren, Bell & Volinsky, “Matrix Factorization Techniques for Recommender Systems,” IEEE Computer, 2009.

What actually won: not one magic algorithm

The winning solution blended many predictors

BellKor’s Pragmatic Chaos

The final winner was a coalition that combined teams and models. The story is as much about collaboration and ensembling as about matrix factorization.



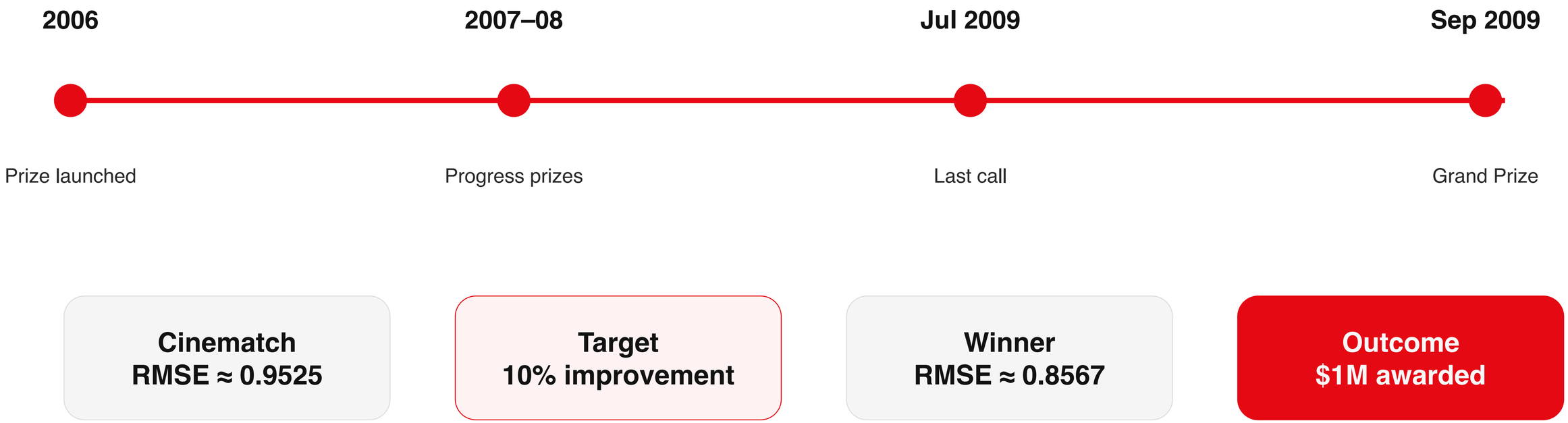
Key lesson

Industrial recommender performance often comes from combining many weak-to-strong signals, not from a single model family.

Sources: Koren (2009), The BellKor Solution to the Netflix Grand Prize; Wired (2009).

Competition results: the last mile was hard

Small RMSE improvements required enormous collaboration



Sources: Netflix Prize Rules; Netflix Prize public result summaries; Wired (2009).

A surprise ending: winning ≠ deployment

The business environment changed while the competition ran

Why the winning algorithm was not simply plugged in

Netflix later said the full winning solution was not deployed largely because the engineering cost was high and the business shifted from DVD ratings toward streaming behavior.

Engineering complexity
Many blended models

Behavior changed
Streaming created richer signals

Metric mismatch
RMSE ≠ total product value

Teaching point

A benchmark victory is not the same as a deployable product decision.

Source: Wired (2012), “Netflix Never Used Its $1 Million Algorithm Due To Engineering Costs.”

Lessons for today’s recommender systems

What students should take from the Netflix Prize

1

Latent factors are powerful

Matrix factorization compresses sparse interactions into useful user/item representations.

2

Biases matter

Global average, user bias, item bias, and temporal effects can explain a lot before personalization.

3

Ensembles win benchmarks

Combining many models can squeeze out performance, but increases complexity.

4

Evaluation defines success

RMSE was measurable and fair, but not a complete measure of product quality.

Sources: Koren, Bell & Volinsky (2009); Koren (2009); Wired (2012).

Discussion prompts for class

Use the Netflix Prize as a bridge from algorithms to product decisions

1. If RMSE improves, does the user experience necessarily improve?
2. Would you deploy a very accurate ensemble if it is difficult to maintain?
3. What metrics would Netflix use today for streaming recommendations?
4. How would you combine matrix factorization with content-based or contextual data?

Class activity: ask students to justify a deployment decision using algorithmic, engineering, and business arguments.

References

Suggested sources for the Matrix Factorization session

1. Netflix Prize Rules. “The Netflix Prize Rules.”

2. Koren, Y., Bell, R., & Volinsky, C. (2009). “Matrix Factorization Techniques for Recommender Systems.” IEEE Computer, 42(8), 30–37.

3. Koren, Y. (2009). “The BellKor Solution to the Netflix Grand Prize.”

4. Wired (2009). “BellKor’s Pragmatic Chaos Wins $1 Million Netflix Prize by Mere Minutes.”

5. Wired (2012). “Netflix Never Used Its $1 Million Algorithm Due To Engineering Costs.”

6. Harvard Business School case abstract: “Netflix: Designing the Netflix Prize (A).”

Use in class: the Netflix Prize is a compact case for latent factors, ensembles, metric design, crowdsourcing, and deployment trade-offs.

Recommender Systems

Matrix Factorisation

Prof. Marc Torrens, PhD
www.marctorrens.com
Associated Professor @ Esade
Board Member @ Medidedalia
Cofounder @ Strands